Bài 7: Lệnh gọi chương trình con, truyền tham số sử dụng ngăn xếp

Họ và tên: Nguyễn Hồng Khôi

Mssv:202416956

ASSIGNMENT 1

Nhập chương trình:

```
# Laboratory Exercise 7 Assignment 1
.text
main:
li a0, -50          # Vị trí(1): a0 là biến cần tìm trị tuyệt đối
jal abs             # Vị trí(2): nhảy đến abs
li a7, 10
ecall
end_main:

abs:
sub s0, zero, a0    # Vị trí (3): s0 là biến lưu giữ giá trị -a0
blt a0, zero, done  # Vị trí (4): so sánh s0 với a0, nếu a0 < s0 thì kết thúc chương trình
add s0, a0, zero    # Vị trí (5): còn nếu a0 > s0 thì gán s0 bằng a0, kết thúc chương trình
done:
jr ra
```

Giải thích:

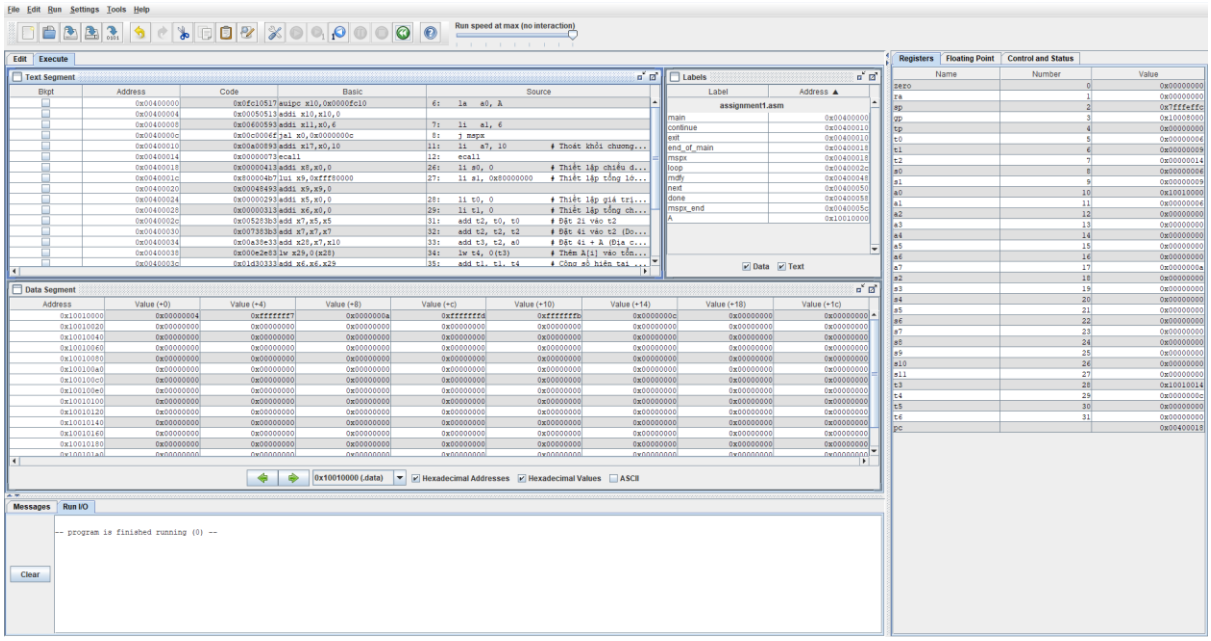
No.	Vị trí	a0	s0	ra
1	(1)	0xffffffce	0	0
2	(2)	0xffffffce	0	0x00400008
3	(3)	0xffffffce	0x00000032	0x00400008
4	---			

<u>Chú thích:</u>

- s0 là thanh ghi chứa kết quả

- a0 là thanh ghi chứa biến

Kết quả sau khi thực thi dòng lệnh:



ASSIGNMENT 2:

Nhập chương trình:

```
# Laboratory Exercise 7, Assignment 2
.text
main:
li a0, 5          # Vị trí (1): gán các giá trị lần lượt vào a0, a1, a2
li a1, 2
li a2, 10
jal max                         # Vị trí (2): nhảy đến max
li a7, 10
ecall
end_main:

max:
add s0, a0, zero      # Vị trí (3): Đặt s0 bằng a0, coi nó đang là lớn nhất
sub t0, a1, s0                  # Vị trí (4): Tính hiệu phần tử thứ 2 với thứ nhất
blt t0, zero, okay    # Vị trí (5): Nếu hiệu này lớn hơn 0 thì cập nhật lại
add s0, a1, zero      # Vị trí (6):
okay:
sub t0, a2, s0                  # Vị trí (7): Thực hiện tương tự như (4)
blt t0, zero, done    # Vị trí (8):
add s0, a2, zero      # Vị trí (9):
done:
```

jr ra

Giải thích:

No.	Vị trí	s0	t0	ra
1	(1)	0	0	0
2	(2)	0	0	0x00400010
3	(3)	0x00000005	0	0x00400010
4	(4)	0x00000005	0xfffffffd	0x00400010
5	(6)	0x00000005	0xfffffffd	0x00400010
6	(7)	0x00000006	0x00000005	0x00400010
7	(9)	0x0000000a	0x00000005	0x00400010

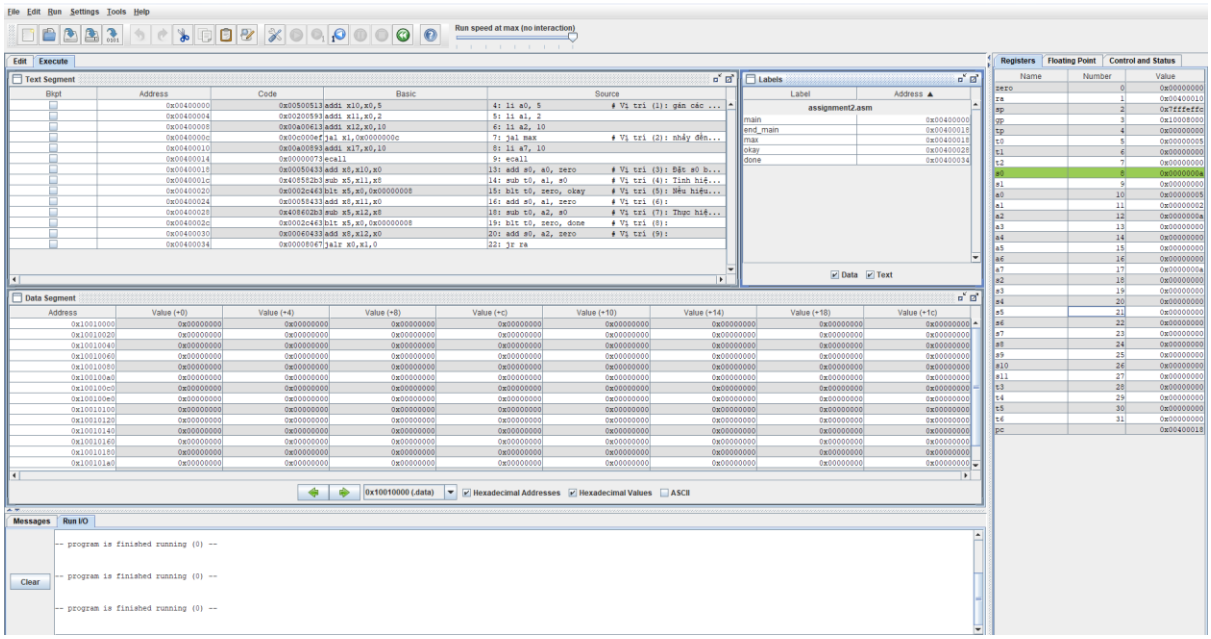
<u>Chú thích:</u>

- a0, a1, a2: lần lượt là vị trí phần tử 1, 2, 3

- s0 : là kết quả tìm max

- t0 : thanh ghi trung gian dùng để so sánh

Kết quả:



ASSIGNMENT 3:

Nhập chương trình:

```
# Laboratory Exercise 7, Assignment 3
.text
push:
addi sp, sp, -8                # Vị trí (1): dịch chuyển con trỏ sp -> tạo không gian cho s0 và s1
li s0, 3          # Vị trí (2): lưu trữ giá trị s
li s1, 5
sw s0, 4(sp)          # Vị trí (3): lưu giá trị của từng biến s vào địa chỉ bộ nhớ
sw s1, 0(sp)
work:
nop                   # nop dùng để mô phỏng lại có đoạn code nào đó nằm ở đây
nop                   # Thực ra là add zero, zero, zero :)))))
nop
pop:
lw s0, 0(sp)          # Vị trí (3): load giá trị
lw s1, 4(sp)
addi sp, sp, 8        # Vị trí (4): free
```

Giải thích:

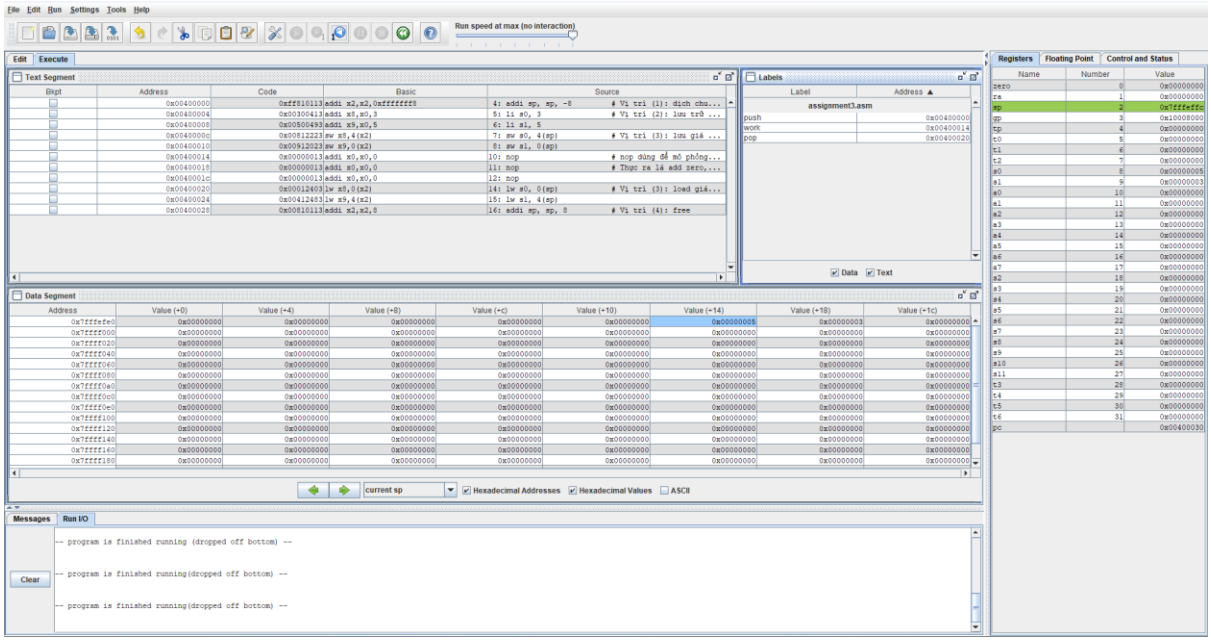
Nol.	Vị trí	sp	s0	s1
1	(1)	0x7fffeff4	0	0
2	(2)	0x7fffeff4	0x00000003	0x00000005
3	(3)	0x7fffeff4	0x00000005	0x00000003
4	(4)	0x7fffeffc	0x00000005	0x00000003

<u>Chú thích:</u>

- s0: lưu trữ giá trị của biến đầu tiên

- s1: lưu trữ giá trị của biến thứ hai

Kết quả:



ASSIGNMENT 4:

Nhập chương trình:

```
# Laboratory Exercise 7, Assignment 4
.data
message: .asciz "Ket qua tinh giai thua la: "
.text
main:
jal WARP                # Vị trí (1): Nhảy đến WARP, lưu địa chỉ caller vào ra
print:
add a1, s0, zero
li a7, 56
la a0, message
ecall
quit:
li a7, 10
ecall
end_main:
WARP:
addi sp, sp, -4                 # Vị trí (2): Cấp phát bộ nhớ
sw ra, 0(sp)            # Vị trí (3): Lưu trữ ra vào địa chỉ
li a0, 2          # Vị trí (4): Lưu trữ N vào a0
jal FACT                # Vị trí (5): nhảy và link đến FACT
lw ra, 0(sp)            # Vị trí (Y): quay trở lại main
addi sp, sp, 4
```

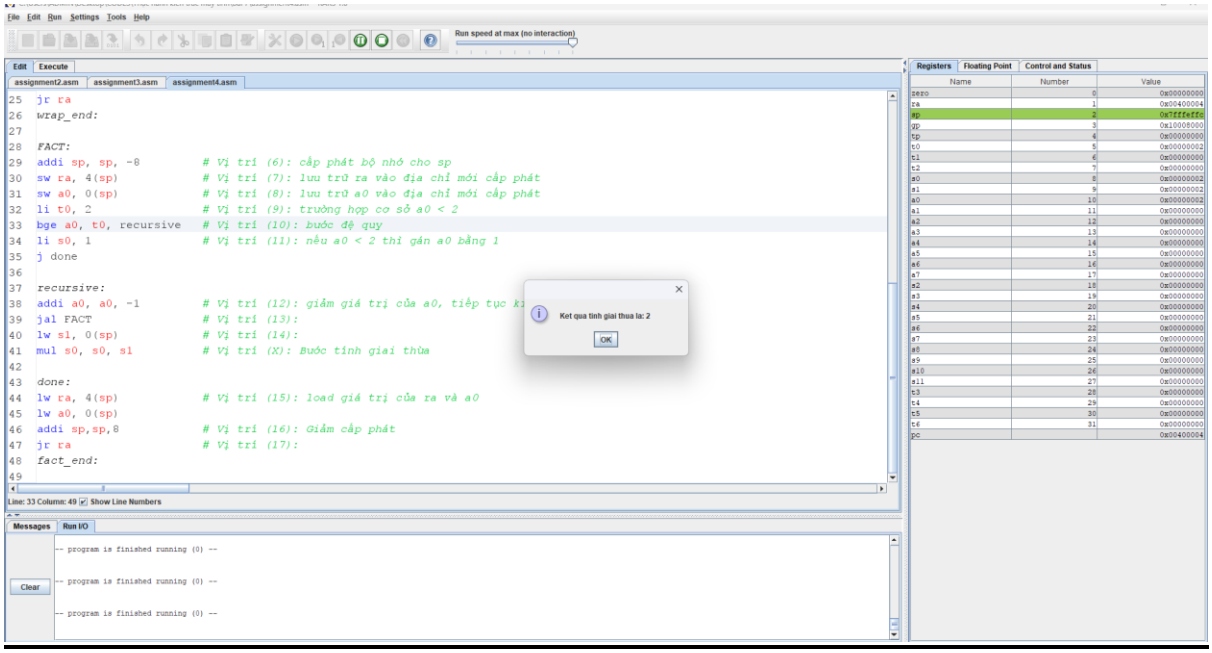
```
jr ra
wrap_end:
FACT:
addi sp, sp, -8                # Vị trí (6): cấp phát bộ nhớ cho sp
sw ra, 4(sp)          # Vị trí (7): lưu trữ ra vào địa chỉ mới cấp phát
sw a0, 0(sp)          # Vị trí (8): lưu trữ a0 vào địa chỉ mới cấp phát
li t0, 2          # Vị trí (9): trường hợp cơ sở a0 < 2
bge a0, t0, recursive          # Vị trí (10): bước đệ quy
li s0, 1          # Vị trí (11): nếu a0 < 2 thì gán a0 bằng 1
j done
recursive:
addi a0, a0, -1                # Vị trí (12): giảm giá trị của a0, tiếp tục kiểm tra
jal FACT              # Vị trí (13):
lw s1, 0(sp)          # Vị trí (14):
mul s0, s0, s1                 # Vị trí (X): Bước tính giai thừa
done:
lw ra, 4(sp)          # Vị trí (15): load giá trị của ra và a0
lw a0, 0(sp)
addi sp,sp,8          # Vị trí (16): Giảm cấp phát
jr ra                 # Vị trí (17):
fact_end:
```

Giải thích:

Nol	Vị trí	sp	ra	a0	t0	s0	s1
1	(1)	0x7fffeffc	0x00400004	0	0	0	0
2	(2)	0x7fffeff8	0x00400004	0	0	0	0
3	(3)	Lưu trữ địa chỉ ra_1 vào 0x7fffeff8					
4	(4)	0x7fffeff8	0x00400004	0x00000002	0	0	0
5	(5)	0x7fffeff8	0x00400030	0x00000002	0	0	0
6	(6)	0x7fffeff0	0x00400030	0x00000002	0	0	0
7	(7), (8)	Lưu trữ ra_2 vào 0x7fffeff4 và và $a0_1$ vào 0x7fffeff0					
8	(9)	0x7fffeff0	0x00400030	0x00000002	0x00000002	0	0
9	(12)	0x7fffeff0	0x00400030	0x00000001	0x00000002	0	0
10	(13)	0x7fffeff0	0x00400060	0x00000001	0x00000002	0	0
11	(6)	0x7fffefe8	0x00400060	0x00000001	0x00000002	0	0

12	(7), (8)	Lưu trữ ra_3 vào 0x7fffefec và và $a0_2$ vào 0x7fffefe8					
13	(9)	0x7fffefe8	0x00400060	0x00000001	0x00000002	0	0
14	(11)	0x7fffefe8	0x00400060	0x00000001	0x00000002	0x00000001	0
15	(15)	0x7fffefe8	0x00400060	0x00000001	0x00000002	0x00000001	0
16	(16)	0x7fffeff0	0x00400060	0x00000001	0x00000002	0x00000001	0
17	(14)	0x7fffeff0	0x00400060	0x00000001	0x00000002	0x00000001	0x00000002
18	(X)	0x7fffeff0	0x00400060	0x00000001	0x00000002	0x00000002	0x00000002
19	(15)	0x7fffeff0	0x00400030	0x00000002	0x00000002	0x00000002	0x00000002
20	(16)	0x7fffeff8	0x00400030	0x00000002	0x00000002	0x00000002	0x00000002
21	(Y)	0x7fffeffc	0x00400004	0x00000002	0x00000002	0x00000002	0x00000002

Kết quả :



ASSIGNMENT 5:

Nhập chương trình:

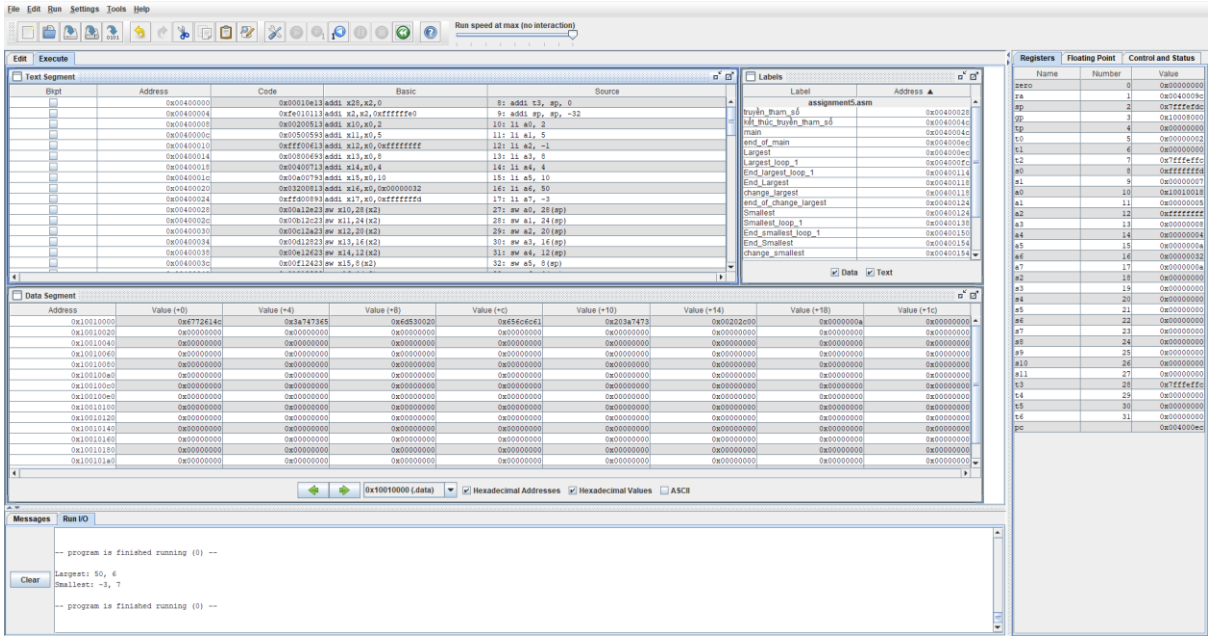
```
# Laboratory Exercise 7, Assignment 5
.data
String_1: .asciz "Largest: "
String_2: .asciz "Smallest: "
String_3: .asciz ", "
String_4: .asciz "\n"
.text
addi t3, sp, 0
addi sp, sp, -32
li a0, 2
li a1, 5
li a2, -1
li a3, 8
li a4, 4
li a5, 10
li a6, 50
li a7, -3

# Param[out]: s0, biến chỉ maximun value
# Param[out]: s1, biến chỉ vị trí của giá trị cần tìm
# Param[in]: t0, biến chỉ giá trị phần tử hiện tại
# Param[in]: t1, biến chỉ vị trí phần tử hiện tại
```

```
# Param[in]: t2, biến tạm thời lưu trữ sp
# Param[in]: t3, lưu trữ địa chỉ điều kiện dừng

truyền_tham_số:
sw a0, 28(sp)
sw a1, 24(sp)
sw a2, 20(sp)
sw a3, 16(sp)
sw a4, 12(sp)
sw a5, 8(sp)
sw a6, 4(sp)
sw a7, 0(sp)
add t2, sp, zero
kết_thúc_truyền_tham_số:

main:
jal Largest
la a0, String_1
li a7, 4
ecall
add a0, zero, s0
li a7, 1
ecall
la a0, String_3
li a7, 4
ecall
add a0, zero, s1
li a7, 1
ecall
```

```
la a0, String_4
li a7, 4
ecall
jal Smallest
la a0, String_2
li a7, 4
ecall
add a0, zero, s0
li a7, 1
ecall
la a0, String_3
li a7, 4
ecall
add a0, zero, s1
li a7, 1
ecall
la a0, String_4
li a7, 4
ecall

li a7, 10
ecall
end_of_main:

Largest:
lw t0, 0(t2)
add s0, zero, t0
li t1, 7
li s1, 7
```

```
        Largest_loop_1:
        addi t2, t2, 4
        beq t2, t3, End_largest_loop_1
        lw t0, 0(t2)
        addi t1, t1, -1
        bgt t0, s0, change_largest
        j Largest_loop_1
        End_largest_loop_1:
        jr ra
End_Largest:
        change_largest:
        add s0, zero, t0
        add s1, zero, t1
        j Largest_loop_1
        end_of_change_largest:

Smallest:
add t2, sp, zero
lw t0, 0(t2)
add s0, zero, t0
li t1, 7
li s1, 7
        Smallest_loop_1:
        addi t2, t2, 4
        beq t2, t3, End_smallest_loop_1
        lw t0, 0(t2)
        addi t1, t1, -1
        blt t0, s0, change_smallest
        j Smallest_loop_1
```

```
        End_smallest_loop_1:
        jr ra
End_Smallest:
        change_smallest:
        add s0, zero, t0
        add s1, zero, t1
        j Largest_loop_1
        end_of_change_smallest:
```

Giải thích: Vòng lặp Largest_loop và Smallest_loop lặp qua từng phần tử, tìm phần tử lớn nhất / nhỏ nhất.

Kết quả: